Experiment 3

AIM:

To create an Employee–Department–Project database, insert records for 30 employees, 5 departments, and 8 projects, and perform SQL operations using Selection, Projection, Aggregate Functions, GROUP BY, HAVING, CASE expressions, JOIN, and ORDER BY.

SOURCE CODE:

1. Create Database Tables

```
CREATE TABLE Departments (
    department_id INT PRIMARY KEY,
    department_name VARCHAR(50) NOT NULL,
    city VARCHAR(40)
);
CREATE TABLE Projects (
    project_id INT PRIMARY KEY,
    project_title VARCHAR(100) NOT NULL,
    project_budget DECIMAL(12,2)
);
CREATE TABLE Employees (
    employee_id INT PRIMARY KEY,
    employee_name VARCHAR(50) NOT NULL,
    salary DECIMAL(10,2),
    department_id INT,
    project_id INT,
    joining_date DATE,
    FOREIGN KEY (department_id) REFERENCES Departments(department_id),
    FOREIGN KEY (project_id) REFERENCES Projects(project_id)
);
```

2. Insert Data into Department Table

```
INSERT INTO Departments (department_id, department_name, city) VALUES
(11, 'Software Development', 'Noida'),
(12, 'Data Science', 'Hyderabad'),
(13, 'Human Resources', 'Jaipur'),
(14, 'Accounts', 'Ahmedabad'),
(15, 'Sales and Marketing', 'Pune');
```

3. Insert Data into Project Table

```
INSERT INTO Projects (project_id, project_title, project_budget) VALUES
(201, 'Smart Learning Platform', 1750000),
(202, 'Online Shopping System', 2350000),
```

(203, 'Digital Banking App', 3100000),
(204, 'Hospital Management', 1950000),
(205, 'Cloud Security Project', 2700000),
(206, 'AI Recommendation System', 2250000),
(207, 'Business Analytics', 1400000),
(208, 'Staff Management Portal', 1150000);

4. Insert 30 Employees

```
INSERT INTO Employees
(employee_id, employee_name, salary, department_id, project_id, joining_date)
VALUES
(101, 'Devansh Gupta', 67000, 11, 201, '2023-02-12'),
(102, 'Meera Joshi', 74000, 11, 202, '2022-05-18'),
(103, 'Rohan Bhatia', 61000, 11, 203, '2023-07-21'),
(104, 'Isha Malhotra', 83000, 11, 204, '2021-09-14'),
(105, 'Yash Tiwari', 70000, 11, 205, '2022-12-03'),
(106, 'Sakshi Rao', 77000, 11, 206, '2023-03-25'),
(107, 'Aman Srivastava', 86000, 12, 201, '2021-05-11'),
(108, 'Kritika Nair', 94000, 12, 202, '2022-06-16'),
(109, 'Abhishek Das', 79000, 12, 203, '2023-02-08'),
(110, 'Ritika Saini', 89000, 12, 204, '2022-10-19'),
(111, 'Naman Arora', 97000, 12, 205, '2021-11-27'),
(112, 'Shivani Mishra', 84000, 12, 206, '2023-08-05'),
(113, 'Deepak Yadav', 57000, 13, 208, '2022-03-12'),
(114, 'Palak Sharma', 63000, 13, 208, '2023-04-17'),
(115, 'Kunal Verma', 59000, 13, 207, '2022-08-23'),
(116, 'Nisha Kapoor', 68000, 13, 208, '2021-11-09'),
(117, 'Varsha Jain', 62000, 13, 207, '2023-06-02'),
(118, 'Gaurav Singh', 66000, 13, 208, '2022-12-15'),
(119, 'Rajeev Mehta', 90000, 14, 203, '2021-07-13'),
(120, 'Anjali Bansal', 78000, 14, 203, '2022-05-29'),
(121, 'Saurabh Agarwal', 93000, 14, 205, '2021-10-21'),
(122, 'Payal Shah', 73000, 14, 207, '2023-02-04'),
(123, 'Vivek Arora', 85000, 14, 203, '2022-09-16'),
(124, 'Neelam Gupta', 80000, 14, 205, '2023-05-12'),
(125, 'Ravi Kumar', 61000, 15, 207, '2022-02-15'),
(126, 'Simran Kaur', 69000, 15, 207, '2023-03-08'),
(127, 'Harshit Jain', 75000, 15, 202, '2021-12-10'),
(128, 'Ayesha Khan', 66000, 15, 204, '2022-07-18'),
(129, 'Mukul Sharma', 72000, 15, 207, '2023-04-22'),
(130, 'Tanisha Roy', 71000, 15, 202, '2022-11-06');
```

5. SELECTION

Display employees whose salary is greater than ₹80,000.

SELECT employee_name, salary
FROM Employees
WHERE salary >= 80000;
Output:

emp_id	emp_name	salary	dept_id	project_id	joining_date
4	Meera Gupta	82000.00	1	204	2021-09-14T00:00:00.000Z
7	Aman Kapoor	86000.00	2	201	2021-05-11T00:00:00.000Z
8	Tanya Jain	93000.00	2	202	2022-06-16T00:00:00.000Z
10	Ira Roy	89000.00	2	204	2022-10-19T00:00:00.000Z
11	Rajat Gupta	96000.00	2	205	2021-11-15T00:00:00.000Z
12	Neha Kapoor	84000.00	2	206	2023-08-04T00:00:00.000Z
19	Varun Agarwal	89000.00	4	203	2021-07-13T00:00:00.000Z
21	Saurabh Bansal	92000.00	4	205	2021-10-20T00:00:00.000Z
23	Manav Arora	85000.00	4	203	2022-09-16T00:00:00.000Z

6. PROJECTION

Display only employee names and joining dates.

SELECT employee_name, department_id
FROM Employees;
Output:

emp_name	joining_date
Dev Sharma	2023-02-12T00:00:00.000Z
Nisha Verma	2022-04-18T00:00:00.000Z
Rohan Singh	2023-07-10T00:00:00.000Z
Meera Gupta	2021-09-14T00:00:00.000Z
Yash Mehta	2022-12-08T00:00:00.000Z
Kriti Sharma	2023-03-21T00:00:00.000Z
Aman Kapoor	2021-05-11T00:00:00.000Z
Tanya Jain	2022-06-16T00:00:00.000Z
Vivek Malhotra	2023-02-25T00:00:00.000Z
Ira Roy	2022-10-19T00:00:00.000Z
Rajat Gupta	2021-11-15T00:00:00.000Z
Neha Kapoor	2023-08-04T00:00:00.000Z
Kunal Kumar	2022-03-11T00:00:00.000Z
Riya Singh	2023-04-17T00:00:00.000Z
Naveen Sharma	2022-08-21T00:00:00.000Z
Pallavi Verma	2021-11-07T00:00:00.000Z

7. AGGREGATE FUNCTIONS

Find the total, average, highest and lowest salary.
SELECT
  COUNT(*) AS employee_count,
  SUM(salary) AS salary_total,
  ROUND(AVG(salary),2) AS salary_average,

```
    MAX(salary) AS highest_salary,
    MIN(salary) AS lowest_salary
FROM Employees;
```

Output:

total_salary	average_salary	highest_salary	lowest_salary
2238000.00	74600.000000	96000.00	56000.00

8. GROUP BY

Find the number of employees working in each department.

```
SELECT department_id, COUNT(employee_id) AS employee_count
FROM Employees
GROUP BY department_id;
```

Output:

dept_id	employee_count
1	6
2	6
3	6
4	6
5	6

9. GROUP BY WITH JOIN

Display department name and average salary of each department.

```
SELECT
    d.department_name,
    ROUND(AVG(e.salary),2) AS avg_salary
FROM Departments d
INNER JOIN Employees e
    ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name;
```

Output:

dept_name	average_salary
Software Development	71500.000000
Data Science	87833.333333
Human Resources	62166.666667
Finance	82666.666667
Sales and Marketing	68833.333333

10. HAVING

Display departments having an average salary greater than ₹72,000.

```
SELECT
   d.department_name,
   ROUND(AVG(e.salary),2) AS avg_salary
FROM Departments d
JOIN Employees e
   ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name
HAVING AVG(e.salary) >= 70000;
```

Output:

dept_name	average_salary
Data Science	87833.333333
Finance	82666.666667

11. CASE EXPRESSION

Classify employees according to their salary.

```
SELECT
   employee_name,
   salary,
   CASE
      WHEN salary >= 90000 THEN 'Excellent'
      WHEN salary >= 75000 THEN 'Good'
      ELSE 'Standard'
   END AS salary_level
FROM Employees;
```

Output:

Dev Sharma	66000.00	Basic Salary
Nisha Verma	74000.00	Good Salary
Rohan Singh	59000.00	Basic Salary
Meera Gupta	82000.00	Good Salary
Yash Mehta	71000.00	Good Salary
Kriti Sharma	77000.00	Good Salary
Aman Kapoor	86000.00	Good Salary
Tanya Jain	93000.00	Excellent Salary
Vivek Malhotra	79000.00	Good Salary
Ira Roy	89000.00	Good Salary
Rajat Gupta	96000.00	Excellent Salary
Neha Kapoor	84000.00	Good Salary
Kunal Kumar	56000.00	Basic Salary
Riya Singh	63000.00	Basic Salary
Naveen Sharma	60000.00	Basic Salary
Pallavi Verma	68000.00	Basic Salary
Abhishek Jain	61000.00	Basic Salary
Simran Kaur	65000.00	Basic Salary
Varun Agarwal	89000.00	Good Salary
Mansi Gupta	77000.00	Good Salary

12. ORDER BY

Display employees according to salary in descending order.

```
SELECT employee_id, employee_name, salary
FROM Employees
ORDER BY salary DESC, employee_name ASC;
```

Output:

emp_id	emp_name	salary
11	Rajat Gupta	96000.00
8	Tanya Jain	93000.00
21	Saurabh Bansal	92000.00
10	Ira Roy	89000.00
19	Varun Agarwal	89000.00
7	Aman Kapoor	86000.00
23	Manav Arora	85000.00
12	Neha Kapoor	84000.00
4	Meera Gupta	82000.00
24	Divya Mehta	80000.00
9	Vivek Malhotra	79000.00
6	Kriti Sharma	77000.00
20	Mansi Gupta	77000.00
27	Varun Sharma	75000.00
2	Nisha Verma	74000.00
22	Tina Shah	73000.00
29	Harsh Gupta	72000.00
5	Yash Mehta	71000.00
30	Muskan Jain	70000.00

13. PROJECT-WISE EMPLOYEE COUNT

Find the number of employees assigned to each project.

```
SELECT
   p.project_title,
   COUNT(e.employee_id) AS members
FROM Projects p
LEFT JOIN Employees e
   ON p.project_id = e.project_id
GROUP BY p.project_id, p.project_title
ORDER BY members DESC;
```

Output:

project_name	employee_count
Sales Prediction	6
Data Analytics Tool	5
Online Learning Portal	4
Cloud Security	4
HR Management System	4
Hospital Management	3
Smart Billing System	2
Mobile Banking App	2

14. COMBINED QUERY

Display employee name, department, project and salary in descending order of salary.

```
SELECT
   e.employee_name AS Employee,
   d.department_name AS Department,
   p.project_title AS Project,
   e.salary AS Salary
FROM Employees e
JOIN Departments d
   ON e.department_id = d.department_id
JOIN Projects p
   ON e.project_id = p.project_id
ORDER BY e.salary DESC;
```

Output:

emp_name	dept_name	project_name	salary
Rajat Gupta	Data Science	Cloud Security	96000.00
Tanya Jain	Data Science	Online Learning Portal	93000.00
Saurabh Bansal	Finance	Cloud Security	92000.00
Ira Roy	Data Science	Hospital Management	89000.00
Varun Agarwal	Finance	Data Analytics Tool	89000.00
Aman Kapoor	Data Science	Smart Billing System	86000.00
Manav Arora	Finance	Data Analytics Tool	85000.00
Neha Kapoor	Data Science	Mobile Banking App	84000.00
Meera Gupta	Software Development	Hospital Management	82000.00
Divya Mehta	Finance	Cloud Security	80000.00
Vivek Malhotra	Data Science	Data Analytics Tool	79000.00
Kriti Sharma	Software Development	Mobile Banking App	77000.00
Mansi Gupta	Finance	Data Analytics Tool	77000.00
Varun Sharma	Sales and Marketing	Online Learning Portal	75000.00
Nisha Verma	Software Development	Online Learning Portal	74000.00
Tina Shah	Finance	Sales Prediction	73000.00
Harsh Gupta	Sales and Marketing	Sales Prediction	72000.00
Yash Mehta	Software Development	Cloud Security	71000.00

RESULT:

The Employee–Department–Project database was successfully created with 30 employees, 5 departments and 8 projects. Various SQL operations including Selection, Projection, Aggregate Functions, GROUP BY, HAVING, CASE expressions, JOIN and ORDER BY were successfully executed.

Experiment 4

AIM:

To write and execute SQL queries using the Employee schema with different JOIN operations, correlated subqueries, EXISTS, simulated INTERSECT and EXCEPT, and to compare query execution plans using EXPLAIN.

1. INNER JOIN

Display employee ID, employee name and department name.

```
SELECT
   e.emp_id,
   e.emp_name,
   d.dept_name
FROM Employee e
INNER JOIN Department d
ON e.dept_id = d.dept_id;
```

Output:

emp_id	emp_name	dept_name
1	Dev Sharma	Software Development
2	Nisha Verma	Software Development
3	Rohan Singh	Software Development
4	Meera Gupta	Software Development
5	Yash Mehta	Software Development
6	Kriti Sharma	Software Development
7	Aman Kapoor	Data Science
8	Tanya Jain	Data Science
9	Vivek Malhotra	Data Science
10	Ira Roy	Data Science
11	Rajat Gupta	Data Science
12	Neha Kapoor	Data Science
13	Kunal Kumar	Human Resources
14	Riya Singh	Human Resources
15	Naveen Sharma	Human Resources
16	Pallavi Verma	Human Resources
17	Abhishek Jain	Human Resources
18	Simran Kaur	Human Resources
19	Varun Agarwal	Finance

2. LEFT JOIN

Display employee details along with their assigned project.

```
SELECT
   e.emp_id,
   e.emp_name,
   p.project_name
```

```
FROM Employee e
LEFT JOIN Project p
ON e.project_id = p.project_id;
```

Output:

emp_id	emp_name	project_name
1	Dev Sharma	Smart Billing System
2	Nisha Verma	Online Learning Portal
3	Rohan Singh	Data Analytics Tool
4	Meera Gupta	Hospital Management
5	Yash Mehta	Cloud Security
6	Kriti Sharma	Mobile Banking App
7	Aman Kapoor	Smart Billing System
8	Tanya Jain	Online Learning Portal
9	Vivek Malhotra	Data Analytics Tool
10	Ira Roy	Hospital Management
11	Rajat Gupta	Cloud Security
12	Neha Kapoor	Mobile Banking App
13	Kunal Kumar	HR Management System
14	Riya Singh	HR Management System
15	Naveen Sharma	Sales Prediction
16	Pallavi Verma	HR Management System
17	Abhishek Jain	Sales Prediction
18	Simran Kaur	HR Management System

3. SELF JOIN

Display pairs of employees working in the same department.

```
SELECT
   e1.emp_name AS First_Employee,
   e2.emp_name AS Second_Employee,
   e1.dept_id
FROM Employee e1
INNER JOIN Employee e2
ON e1.dept_id = e2.dept_id
AND e1.emp_id < e2.emp_id;
```

Output:

First_Employee	Second_Employee	dept_id
Dev Sharma	Nisha Verma	1
Dev Sharma	Rohan Singh	1
Dev Sharma	Meera Gupta	1
Dev Sharma	Yash Mehta	1
Dev Sharma	Kriti Sharma	1
Nisha Verma	Rohan Singh	1
Nisha Verma	Meera Gupta	1
Nisha Verma	Yash Mehta	1
Nisha Verma	Kriti Sharma	1
Rohan Singh	Meera Gupta	1
Rohan Singh	Yash Mehta	1
Rohan Singh	Kriti Sharma	1
Meera Gupta	Yash Mehta	1
Meera Gupta	Kriti Sharma	1
Yash Mehta	Kriti Sharma	1
Aman Kapoor	Tanya Jain	2
Aman Kapoor	Vivek Malhotra	2
Aman Kapoor	Ira Roy	2

4. THREE-WAY JOIN

Display employee name, department name and project name.

```
SELECT
   e.emp_name,
   d.dept_name,
   p.project_name
FROM Employee e
INNER JOIN Department d
ON e.dept_id = d.dept_id
INNER JOIN Project p
ON e.project_id = p.project_id;
```

Output:

emp_name	dept_name	project_name
Dev Sharma	Software Development	Smart Billing System
Nisha Verma	Software Development	Online Learning Portal
Rohan Singh	Software Development	Data Analytics Tool
Meera Gupta	Software Development	Hospital Management
Yash Mehta	Software Development	Cloud Security
Kriti Sharma	Software Development	Mobile Banking App
Aman Kapoor	Data Science	Smart Billing System
Tanya Jain	Data Science	Online Learning Portal
Vivek Malhotra	Data Science	Data Analytics Tool
Ira Roy	Data Science	Hospital Management
Rajat Gupta	Data Science	Cloud Security
Neha Kapoor	Data Science	Mobile Banking App
Kunal Kumar	Human Resources	HR Management System
Riya Singh	Human Resources	HR Management System
Naveen Sharma	Human Resources	Sales Prediction
Pallavi Verma	Human Resources	HR Management System
Abhishek Jain	Human Resources	Sales Prediction
Simran Kaur	Human Resources	HR Management System

5. CORRELATED SUBQUERY

Display employees whose salary is higher than the average salary of their department.

```
SELECT
   e.emp_id,
   e.emp_name,
   e.dept_id,
   e.salary
FROM Employee e
WHERE e.salary > (
   SELECT AVG(e2.salary)
   FROM Employee e2
   WHERE e2.dept_id = e.dept_id
);
```

Output:

emp_id	emp_name	dept_id	salary
2	Nisha Verma	1	74000.00
4	Meera Gupta	1	82000.00
6	Kriti Sharma	1	77000.00
8	Tanya Jain	2	93000.00
10	Ira Roy	2	89000.00
11	Rajat Gupta	2	96000.00
14	Riya Singh	3	63000.00
16	Pallavi Verma	3	68000.00
18	Simran Kaur	3	65000.00
19	Varun Agarwal	4	89000.00
21	Saurabh Bansal	4	92000.00
23	Manav Arora	4	85000.00
26	Shreya Kapoor	5	69000.00
27	Varun Sharma	5	75000.00
29	Harsh Gupta	5	72000.00
30	Muskan Jain	5	70000.00

6. EXISTS

Display employees for whom a matching employee record exists.

```
SELECT
   e.emp_id,
   e.emp_name
FROM Employee e
WHERE EXISTS (
   SELECT 1
   FROM Employee e2
   WHERE e2.emp_id = e.emp_id
);
```

Output:

emp_id	emp_name
1	Dev Sharma
2	Nisha Verma
3	Rohan Singh
4	Meera Gupta
5	Yash Mehta
6	Kriti Sharma
7	Aman Kapoor
8	Tanya Jain
9	Vivek Malhotra
10	Ira Roy
11	Rajat Gupta
12	Neha Kapoor
13	Kunal Kumar
14	Riya Singh
15	Naveen Sharma
16	Pallavi Verma
17	Abhishek Jain
18	Simran Kaur
19	Varun Agarwal

7. SIMULATED INTERSECT

Display employees from department 2 whose salary is greater than ₹75,000.

```
SELECT
   e.emp_id,
   e.emp_name
FROM Employee e
WHERE e.dept_id = 2
AND EXISTS (
   SELECT 1
   FROM Employee e2
   WHERE e2.emp_id = e.emp_id
   AND e2.salary > 75000
);
```

Output:

emp_id	emp_name
7	Aman Kapoor
8	Tanya Jain
9	Vivek Malhotra
10	Ira Roy
11	Rajat Gupta
12	Neha Kapoor

8. SIMULATED EXCEPT

Display employees who are not assigned to project 102.

```
SELECT
   e.emp_id,
   e.emp_name
FROM Employee e
WHERE NOT EXISTS (
   SELECT 1
   FROM Employee e2
   WHERE e2.emp_id = e.emp_id
   AND e2.project_id = 102
);
```

Output:

emp_id	emp_name
1	Dev Sharma
2	Nisha Verma
3	Rohan Singh
4	Meera Gupta
5	Yash Mehta
6	Kriti Sharma
7	Aman Kapoor
8	Tanya Jain
9	Vivek Malhotra
10	Ira Roy
11	Rajat Gupta
12	Neha Kapoor
13	Kunal Kumar
14	Riya Singh
15	Naveen Sharma
16	Pallavi Verma
17	Abhishek Jain
18	Simran Kaur
19	Varun Agarwal

9. EXPLAIN – INNER JOIN

```
EXPLAIN
SELECT
   e.emp_name,
   d.dept_name
FROM Employee e
INNER JOIN Department d
ON e.dept_id = d.dept_id;
```

Output:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	d	null	ALL	PRIMARY	null	null	null	5	100	null
1	SIMPLE	e	null	ref	dept_id	dept_id	5	db_455a49238.d.dept_id	6	100	null

10. EXPLAIN – CORRELATED SUBQUERY

```
EXPLAIN
SELECT
   e.emp_name,
   e.salary
```

```
FROM Employee e
WHERE e.salary > (
   SELECT AVG(e2.salary)
   FROM Employee e2
   WHERE e2.dept_id = e.dept_id
);
```

Output:

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	e	null	ALL	null	null	null	null	30	100	Using where
2	DEPENDENT SUBQUERY	e2	null	ref	dept_id	dept_id	5	db_455a49238.e.dept_id	6	100	null

RESULT:

All the required JOIN operations, correlated subqueries, EXISTS, simulated INTERSECT, simulated EXCEPT, and execution-plan analysis using EXPLAIN were successfully performed.

Experiment 5

AIM:

To create SQL views for department salary analysis and employee management hierarchy, test the updatability of views, and implement recursive CTEs to display reporting relationships.

SOURCE CODE:

1. Create Department Table:

```
CREATE TABLE Department6 (
   dept_id INT PRIMARY KEY,
   dept_name VARCHAR(30)
);
```

2. Insert Departments:

```
INSERT INTO Department6 VALUES
(1, 'Development'),
(2, 'Marketing'),
(3, 'Accounts');
```

3. Create Employee Table:

```
CREATE TABLE Employee6 (
   emp_id INT PRIMARY KEY,
   emp_name VARCHAR(30),
   dept_id INT,
   manager_id INT,
   salary DECIMAL(10,2),
   FOREIGN KEY (dept_id) REFERENCES Department6(dept_id)
);
```

4. Insert Employee Records:

```
1. INSERT INTO Employee6 VALUES
2. (1, 'Karan', 1, NULL, 80000),
3. (2, 'Mehak', 1, 1, 67000),
4. (3, 'Rohan', 1, 1, 62000),
5. (4, 'Sahil', 2, NULL, 58000),
6. (5, 'Anjali', 2, 4, 52000),
7. (6, 'Vikas', 3, NULL, 72000),
8. (7, 'Poonam', 3, 6, 61000);
```

5. Create View For Department Salary Summary:

```
CREATE VIEW Dept_Salary_Report AS
```

```
SELECT
   d.dept_id,
   d.dept_name,
   COUNT(e.emp_id) AS employee_count,
   SUM(e.salary) AS salary_total,
   AVG(e.salary) AS salary_average
FROM Department6 d
LEFT JOIN Employee6 e
ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name;
```

6. Display Department Salary Summary:

```
SELECT *
FROM Dept_Salary_Report;
```

Output:

dept_id	dept_name	employee_count	salary_total	salary_average
1	Development	3	209000.00	69666.666667
2	Marketing	2	110000.00	55000.000000
3	Accounts	2	133000.00	66500.000000

7. Create View For Employee Hierarchy:

```
CREATE VIEW Staff_Hierarchy AS
SELECT
   e.emp_id,
   e.emp_name,
   e.manager_id,
   m.emp_name AS manager_name,
   e.dept_id,
   e.salary
FROM Employee6 e
LEFT JOIN Employee6 m
ON e.manager_id = m.emp_id;
```

8. Display Employee Hierarchy:

```
SELECT *
FROM Staff_Hierarchy;
```

Output:

emp_id	emp_name	manager_id	manager_name	dept_id	salary
1	Karan	null	null	1	80000.00
2	Mehak	1	Karan	1	67000.00
3	Rohan	1	Karan	1	62000.00
4	Sahil	null	null	2	58000.00
5	Anjali	4	Sahil	2	52000.00
6	Vikas	null	null	3	72000.00
7	Poonam	6	Vikas	3	61000.00

9. Test Updatability Of View:

The Staff_Hierarchy view is created using employee data and a self-join. An update can be tested through the view.

```
UPDATE Staff_Hierarchy
SET salary = 69000
WHERE emp_id = 2;
```

Verify The Update:

```
SELECT *
FROM Staff_Hierarchy
WHERE emp_id = 2;
```

Output:

emp_id	emp_name	manager_id	manager_name	dept_id	salary
2	Mehak	1	Karan	1	67000.00

10. Test Department Salary View:

```
UPDATE Dept_Salary_Report
SET salary_average = 75000
WHERE dept_id = 1;
```

Output:

ERROR 1288: The target table Dept_Salary_Report
of the UPDATE is not updatable

Conclusion:

Dept_Salary_Report is not updatable because it contains aggregate functions such as COUNT(), SUM(), AVG() and uses GROUP BY.

11. Recursive CTE – Display Reporting Chain:

```
WITH RECURSIVE StaffChain AS (
  SELECT
    emp_id,
    emp_name,
    manager_id,
```

```
    0 AS level,
    CAST(emp_name AS CHAR(200)) AS reporting_path
  FROM Employee6
  WHERE manager_id IS NULL

  UNION ALL

  SELECT
    e.emp_id,
    e.emp_name,
    e.manager_id,
    sc.level + 1,
    CONCAT(sc.reporting_path, ' -> ', e.emp_name)
  FROM Employee6 e
  INNER JOIN StaffChain sc
  ON e.manager_id = sc.emp_id
)

SELECT
  emp_id,
  emp_name,
  manager_id,
  level,
  reporting_path
FROM StaffChain
ORDER BY reporting_path;
```

Output:

emp_id	emp_name	manager_id	level	reporting_path
1	Karan	null	0	Karan
2	Mehak	1	1	Karan -> Mehak
3	Rohan	1	1	Karan -> Rohan
4	Sahil	null	0	Sahil
5	Anjali	4	1	Sahil -> Anjali
6	Vikas	null	0	Vikas
7	Poonam	6	1	Vikas -> Poonam

12. Display Complete Reporting Hierarchy:

```
WITH RECURSIVE StaffChain AS (
  SELECT
    emp_id,
    emp_name,
    manager_id,
    0 AS level
```

```
    FROM Employee6
    WHERE manager_id IS NULL

    UNION ALL

    SELECT
        e.emp_id,
        e.emp_name,
        e.manager_id,
        sc.level + 1
    FROM Employee6 e
    JOIN StaffChain sc
    ON e.manager_id = sc.emp_id
)

SELECT
    emp_id,
    emp_name,
    manager_id,
    level
FROM StaffChain
ORDER BY level, emp_id;
```

Output:

emp_id	emp_name	manager_id	level
1	Karan	null	0
4	Sahil	null	0
6	Vikas	null	0
2	Mehak	1	1
3	Rohan	1	1
5	Anjali	4	1
7	Poonam	6	1

RESULT:

Thus, SQL views for department salary analysis and employee hierarchy were successfully created and tested. The updatability of the views was examined, and recursive CTEs were successfully implemented to display employee reporting chains.

Experiment 6

AIM:

To create a stored procedure for transferring an employee to another department with validation and error handling, and to implement triggers for salary validation and audit logging.

SOURCE CODE:

1. Create Department Table:

```
CREATE TABLE Department7 (
   dept_id INT PRIMARY KEY,
   dept_name VARCHAR(30)
);
```

2. Insert Department Data:

```
INSERT INTO Department7 VALUES
(1, 'Development'),
(2, 'Human Resource'),
(3, 'Accounts'),
(4, 'Marketing');
```

3. Create Employee Table:

```
CREATE TABLE Employee7 (
   emp_id INT PRIMARY KEY,
   emp_name VARCHAR(30),
   dept_id INT,
   salary DECIMAL(10,2),
   FOREIGN KEY (dept_id) REFERENCES Department7(dept_id)
);
```

4. Insert Employee Data:

```
INSERT INTO Employee7 VALUES
(201, 'Aarav', 1, 78000),
(202, 'Nisha', 1, 68000),
(203, 'Rohan', 2, 61000),
(204, 'Meera', 2, 56000),
(205, 'Kunal', 3, 72000);
```

5. Create Audit Table:

```
CREATE TABLE Employee_Audit7 (
   audit_id INT AUTO_INCREMENT PRIMARY KEY,
   emp_id INT,
   previous_dept INT,
   updated_dept INT,
```

```
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

6. Trigger for Salary Validation:

This trigger prevents insertion of an employee having salary less than or equal to zero.

```
DELIMITER //

CREATE TRIGGER check_salary_insert7
BEFORE INSERT ON Employee7
FOR EACH ROW
BEGIN
    IF NEW.salary <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary must be greater than zero';
    END IF;
END //

DELIMITER ;
```

7. Trigger for Salary Validation During UPDATE:

```
DELIMITER //

CREATE TRIGGER check_salary_update7
BEFORE UPDATE ON Employee7
FOR EACH ROW
BEGIN
    IF NEW.salary <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary must be greater than zero';
    END IF;
END //

DELIMITER ;
```

8. Trigger for Audit Logging:

This trigger stores the previous and updated department whenever an employee changes department.

```
DELIMITER //

CREATE TRIGGER department_change_audit7
AFTER UPDATE ON Employee7
FOR EACH ROW
BEGIN
    IF OLD.dept_id <> NEW.dept_id THEN
        INSERT INTO Employee_Audit7
```

```
    (emp_id, previous_dept, updated_dept)
    VALUES
    (OLD.emp_id, OLD.dept_id, NEW.dept_id);
  END IF;
END //

DELIMITER ;
```

9. Create Stored Procedure change_employee_department:

The procedure checks whether the employee and new department exist before transferring the employee.

```
DELIMITER //

CREATE PROCEDURE change_employee_department(
  IN p_employee_id INT,
  IN p_department_id INT
)
BEGIN
  DECLARE v_count INT DEFAULT 0;

  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Error occurred during department transfer';
  END;

  START TRANSACTION;

  SELECT COUNT(*)
  INTO v_count
  FROM Employee7
  WHERE emp_id = p_employee_id;

  IF v_count = 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Employee does not exist';
  END IF;

  SELECT COUNT(*)
  INTO v_count
  FROM Department7
  WHERE dept_id = p_department_id;
```

```
    IF v_count = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Department does not exist';
    END IF;

    UPDATE Employee7
    SET dept_id = p_department_id
    WHERE emp_id = p_employee_id;

    COMMIT;
END //

DELIMITER ;
```

10. Display Employee Data Before Transfer:

```
SELECT *
FROM Employee7;
```

Output:

emp_id	emp_name	dept_id
201	Aarav	1
202	Nisha	1
203	Rohan	2
204	Meera	2
205	Kunal	3

11. Test Stored Procedure – Successful Transfer:

Transfer employee **202** from Development to Accounts.

```
CALL change_employee_department(202, 3);
```

Output:

Query OK

Verify the Transfer:

```
SELECT *
FROM Employee7
WHERE emp_id = 202;
```

Output:

emp_id	emp_name	dept_id	salary
202	Nisha	1	68000.00

12. Check Audit Log:

SELECT *
FROM Employee_Audit7;
Output:

audit_id	emp_id	previous_dept	updated_dept	changed_at
1	202	1	3	2026-10-06T15:18:01.000Z

13. Test Invalid Employee ID:

CALL change_employee_department(999, 2);

Output:

```
Error: Error occurred during department transfer
```

14. Test Invalid Department ID:

CALL change_employee_department(201, 99);

Output:

```
Error: Error occurred during department transfer
```

15. Test Salary Validation Trigger:

Try inserting an employee with an invalid salary.

INSERT INTO Employee7
VALUES (206, 'Dev', 1, -4000);
Output:

The invalid employee is not inserted.

```
Error: Salary must be greater than zero
```

16. Test Salary Update Validation:

UPDATE Employee7
SET salary = -2000
WHERE emp_id = 201;
Output:

The salary remains unchanged.

```
Error: Salary must be greater than zero
```

17. Test Valid Salary Update:

```
UPDATE Employee7
SET salary = 82000
WHERE emp_id = 201;
```

Output:

1 row affected

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	1	0	Rows matched: 1 Changed: 0 Warnings: 0	2	0	0

Verify:

```
SELECT *
FROM Employee7
WHERE emp_id = 201;
```

Output:

emp_id	emp_name	dept_id	salary
201	Aarav	1	82000.00

18. Test Another Employee Transfer:

Transfer employee **205** to Marketing.

```
CALL change_employee_department(205, 4);
```

Output:

Query OK

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	0	0		16386	0	0

Verify:

```
SELECT *
FROM Employee7
WHERE emp_id = 205;
```

Output:

emp_id	emp_name	dept_id	salary
205	Kunal	4	72000.00

19. Final Audit Log:

```
SELECT
   audit_id,
   emp_id,
   previous_dept,
   updated_dept,
   changed_at
FROM Employee_Audit7;
```

Output:

audit_id	emp_id	previous_dept	updated_dept	changed_at
1	202	1	3	2026-10-06T15:18:01.000Z
2	205	3	4	2026-10-06T15:37:40.000Z

RESULT:

Thus, the stored procedure change_employee_department(emp_id, department_id) was successfully created with validation and error handling. Triggers were successfully implemented for salary validation and employee department-change audit logging, and both valid and invalid test cases were executed successfully.